

LLM-Guided Synthesis of Constraint-Adaptive Heuristics for Satellite–Ground Scheduling

Anonymous Author(s)

Submission Id: 1997

Abstract

Graph optimization heuristics can achieve strong aggregate performance while relying on decision rules that fail under changes in resource constraints. Effective adaptation requires identifying not only when action preferences should change, but also whether the input representation contains the information needed for that change. We propose an LLM-guided synthesis framework for constraint-adaptive satellite–ground scheduling that uses certified decision evidence to jointly refine graph representations and ranking rules. Its central mechanism detects ranking requirements that become contradictory under the current feature representation and directs synthesis toward the missing structural distinctions, rather than repeatedly adjusting an insufficient scoring interface. Two complementary mechanisms support this refinement. First, controlled offline interventions on resource-constraint configurations generate paired static scheduling problems in which the same competing actions remain individually feasible. Sound bounds on conditional completion values certify preference reversals or preservation under explicit boundary commitments, yielding relational specifications of when rankings should change or remain stable. Second, structural witnesses guide the LLM to compose distinguishing features from a typed graph-operation library and synthesize corresponding ranking rules. Feature–rule pairs are selected jointly for specification consistency, complete-schedule quality, and feature-computation cost, making representation expansion targeted and cost-aware. We formulate contact selection as maximum-weight independent set optimization and study adaptation using a frozen program on held-out scheduling instances and constraint configurations. The synthesized program responds to the current conflict graph within a fixed feasibility-enforcing kernel, without online LLM or oracle calls. By linking representation refinement to certified decision conflicts, the framework addresses information limitations beyond rule-only repair while explicitly accounting for the computational cost of constraint adaptation.

CCS Concepts

• **Computing methodologies** → **Planning and scheduling**; • **Mathematics of computing** → *Combinatorial optimization*.

Keywords

Satellite scheduling, maximum-weight independent set, heuristic synthesis, representation refinement, large language models

ACM Reference Format:

Anonymous Author(s). 2027. LLM-Guided Synthesis of Constraint-Adaptive Heuristics for Satellite–Ground Scheduling. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 6 pages.

1 Introduction

Resource constraints change the opportunity cost of selecting a contact in satellite–ground scheduling. Selecting a long contact can exclude several short contacts; whether those alternatives can themselves coexist depends on the current resource-conflict graph. A priority rule that works well on average may make inappropriate decisions on a different constraint configuration. Merely reweighting its existing inputs can fail when those inputs omit a distinction necessary for the desired ordering.

Large language models (LLMs) offer a flexible mechanism for proposing optimization programs. Existing systems evolve code using measured quality, reflection, or formal property feedback [3–5, 7]. These precedents motivate a specific question: when certified decision evidence cannot be expressed by the current observable representation, can that failure guide a targeted expansion of the representation itself?

We investigate paired static scheduling problems. An offline intervention changes one admissible resource parameter while preserving contact opportunities, rewards, and explicit boundary commitments. Conditional completion bounds establish which of two competing actions is preferable on each side. Opposite strict preferences specify a reversal; agreeing strict preferences specify preservation. An inconclusive comparison does not certify preservation.

Our central mechanism projects strict requirements onto exact feature equivalence classes. A directed cycle in this quotient certifies that no deterministic pointwise scalar score of the current representation can satisfy all observed requirements. Structural witnesses expose the lost distinction and guide typed graph-feature proposals. Representation and rule are selected together using relational consistency, complete-schedule reward, and feature cost, then frozen inside a feasibility-enforcing kernel.

Status of this draft. We present an executable framework and a small assisted-synthesis pilot. The current coding assistant proposed part of a fixed candidate bank; there were zero automated LLM API calls. The selected program also occurs in a deterministic enumeration, and shared-bank free-joint selection chooses the same program. The pilot validates the implementation and exposes representation limitations, but does not establish an advantage of LLM generation or contradiction-targeted search. Those claims require independent matched-budget generation experiments.



This work is licensed under a Creative Commons Attribution International 4.0 License.

2 Related Work

FunSearch combines generated code, a program database, and a fixed algorithmic skeleton [5]; EoH evolves ideas alongside code [3]; ReEvo uses short- and long-term reflection to steer evolution [7]. Offline synthesis, a feasible skeleton, and reflective repair are therefore precedents, not our distinguishing claim.

Property-Guided LLM Program Synthesis for Planning uses formal properties and concrete counterexamples to repair heuristic programs [4]. It is a particularly close precedent. Our quotient diagnosis asks whether *any* pointwise score over the current vectors can satisfy the observed strict requirements, whereas a candidate check asks whether a particular program satisfies them. HeurA-genix evolves strategies from contrasting trajectories and supports state-dependent selection [6]. Our labels require separating sound conditional completion intervals under aligned configurations.

DRAGON uses LLM-driven decomposition and local reconstruction on the graph instance being solved [1]. We target offline synthesis and cross-configuration reuse, so discovery and per-instance deployment costs must be separated in a direct comparison. Refining abstractions with counterexamples is established [2]; our finite quotient proposition is a diagnostic tool, not a new general abstraction-refinement theory. Our proposed focus is its connection to certified scheduling relations and cost-aware structural feature refinement.

3 Evidence-Driven Representation and Rule Refinement

We study a family of static satellite–ground scheduling problems under different resource-constraint configurations. Each configuration is fixed during a scheduling run. Offline interventions provide evidence about how decisions should adapt across configurations; deployment uses one frozen program that reads the current conflict graph. Our central distinction is between a rule that misuses available information and a representation that cannot express the required rankings.

3.1 Scheduling Model and Fixed Execution Kernel

Let V be a set of contact opportunities with nonnegative rewards w_v . A legal configuration θ induces a conflict graph $G_\theta = (V, E_\theta, w)$, where an edge denotes joint infeasibility of two contacts under the modelled resource constraints. Contact selection is the maximum-weight independent set problem

$$\max_{I \subseteq V} w(I) = \sum_{v \in I} w_v \quad \text{subject to } I \text{ independent in } G_\theta. \quad (1)$$

This reduction assumes that all hard constraints in the evaluated model are captured by the graph and any explicit boundary commitments. Constraints involving higher-order resource interactions require an extended model rather than an unsupported pairwise-feasibility claim.

A boundary $B = (F, X)$ consists of fixed selected contacts F and explicitly excluded contacts X . We require F to be independent and $F \cap X$ to be empty. With $N_\theta(F)$ denoting the union of the neighbours of F , the residual candidates are

$$R_\theta(B) = V \setminus (F \cup X \cup N_\theta(F)). \quad (2)$$

A program $P = (\phi, h)$ computes features $\phi(G_\theta, R, v)$ and a finite score $h(\phi(G_\theta, R, v))$ for each $v \in R$. The kernel selects a highest-scoring contact, adds it to the schedule, and removes its closed neighbourhood from R . Ties use a fixed deterministic convention; a tie does not satisfy a strict ranking requirement. The final schedule is checked independently against the graph.

Kernel invariant. Starting from a feasible F , each selected contact belongs to the residual set and is therefore compatible with every previously selected contact. Induction preserves independence. At least the selected contact is removed at each step, so the finite process terminates. This establishes feasibility for the graph model, not optimality of the learned ordering.

3.2 Certified Relations from Aligned Interventions

An aligned intervention changes admissible resource parameters from θ_0 to θ_1 , rebuilding conflict edges while preserving contact identities, rewards, objective units, and the intended boundary semantics. The competing actions a and b must be distinct, adjacent in both graphs, and individually feasible under B in both configurations. In particular, we reject comparisons whose apparent preference change is caused by an action becoming infeasible.

Let I_θ be the independent sets of G_θ . For $d \in \{a, b\}$, define the full conditional completion value

$$V_\theta(d \mid B) = \max\{w(I) : I \in I_\theta, F \cup \{d\} \subseteq I, I \cap X = \emptyset\}. \quad (3)$$

The maximization includes all remaining contacts, not only a sampled local region. The certificate scope is conditional on B ; changing a boundary changes the quantity being certified. A checked feasible completion for each action–configuration combination establishes that all four spaces are nonempty.

A budgeted oracle returns sound intervals

$$L_\theta(d \mid B) \leq V_\theta(d \mid B) \leq U_\theta(d \mid B). \quad (4)$$

Lower bounds are values of independently checked full-graph schedules. Upper bounds must bound the entire conditional completion space, for example through a valid relaxation or the open frontier of branch and bound. For an oracle region H , let $A = R_\theta(B \cup \{d\})$, where $B \cup \{d\}$ means adding d to F . If U_H bounds the MWIS value of $G_\theta[A \cap H]$, one valid, possibly loose upper bound is

$$U_\theta(d \mid B) = w(F \cup \{d\}) + U_H + \sum_{v \in A \setminus H} w_v. \quad (5)$$

It relaxes the conflicts involving the outside region. A local feasible solution must be completed and checked globally before its value is used as a lower bound. Expanding H may tighten evidence, but budget exhaustion never turns an unresolved interval into a certified label. Integer or outward-rounded objective arithmetic is used to preserve bound validity.

For an objective tolerance $\epsilon \geq 0$, the inequalities

$$\begin{aligned} L_{\theta_0}(a \mid B) &> U_{\theta_0}(b \mid B) + \epsilon, \\ L_{\theta_1}(b \mid B) &> U_{\theta_1}(a \mid B) + \epsilon \end{aligned} \quad (6)$$

certify a strict preference reversal. They imply the same strict ordering of the respective conditional optima by Equation 4. If the same action satisfies the strict inequality in both configurations,

we certify preference preservation. Preservation concerns ordering, not equality of objective values. Overlapping intervals remain unresolved; a rollout average or an incumbent alone is insufficient to certify the opposite bound.

Each certified comparison induces a relational specification $r = (x^+, x^-)$, where $x = (G_\theta, R_\theta(B), v)$ is a decision occurrence and x^+ is preferred. The requirement is

$$h(\phi(x^+)) > h(\phi(x^-)). \quad (7)$$

Implementation checks may impose a separate numerical score margin. That margin is not the oracle’s objective tolerance ϵ . The evidence certifies a comparison of optimal completions under a specified boundary. It does not guarantee that every heuristic completion following the preferred action dominates every completion following the other action, or that greedy use of all such relations is globally optimal.

3.3 Detecting Representation-Induced Contradictions

Let $\mathcal{D}$ be the finite set of certified requirements and $\mathcal{Z}$ its decision occurrences. The representation includes every value observable by the scoring function. Define exact feature aliasing by

$$x \sim_\phi y \iff \phi(x) = \phi(y). \quad (8)$$

The quotient requirement graph Q_ϕ has one vertex for each equivalence class in $\mathcal{Z}/\sim_\phi$ and an edge $[x^+] \rightarrow [x^-]$ for every requirement. Self-loops are retained. Equality uses exact canonical feature values; clustering or rounded similarity is not used as a proof of aliasing.

Proposition (finite-sample representability). There exists an unrestricted deterministic pointwise real-valued function on the observed feature vectors satisfying all strict requirements if and only if Q_ϕ is acyclic. Thus a cycle or self-loop proves that no rule over this representation can satisfy all requirements.

Proof. Because a pointwise function assigns the same score to identical feature vectors, every edge requires a strict decrease between its endpoint scores. A directed cycle would imply $s_1 > s_2 > \dots > s_k > s_1$, a contradiction; a self-loop requires $s_1 > s_1$. Conversely, in a finite acyclic graph assign to each vertex the length of its longest outgoing path. Every edge then has a larger score at its source than at its destination. These scores define a function on the observed vectors satisfying every requirement. They can be scaled to any fixed finite positive margin. $\square$

The converse is an abstract existence statement: a lookup mapping on a finite set is not necessarily expressible by the permitted rule language, and gives no guarantee on unseen vectors. Therefore an acyclic quotient does not certify success of concrete DSL synthesis. Approximate aliases are useful diagnostics but cannot establish this impossibility result. The proposition applies to the current pointwise scoring interface, not to arbitrary policies allowed to inspect extra graph structure, action identifiers, or history. Instance and action identifiers are excluded from synthesized semantic features to prevent label lookup.

A cycle returns its requirements and the underlying aliased graph occurrences as a structural witness. The refinement loop distinguishes this obstruction from a candidate rule merely failing

an expressible requirement. It first audits intervention alignment and certificate validity, then requests distinguishing features. This makes representation expansion a response to identified missing distinctions rather than an unconditional increase in feature count.

3.4 Typed Structural Features and Offline Synthesis

The feature language distinguishes individual nodes, node sets, edge sets, and numbers. Its operations expose the root contact, the available set, neighborhood queries, typed set algebra, induced edges, counts, and aggregates of contact weights. Edge aggregates include sums of endpoint minimum weights and endpoint weight products, with bounded numerical arithmetic. For example, if $C(v) = R \setminus (\{v\} \cup N_\theta(v))$, both

$$f_1(v) = \sum_{u \in C(v)} w_u, \quad f_2(v) = \sum_{\{u,z\} \in E_\theta[C(v)]} \min(w_u, w_z) \quad (9)$$

are deployable graph statistics. Equal f_1 values can hide different internal compatibility among surviving opportunities; f_2 can expose some such differences. These aggregates are features, not automatically valid completion bounds. In particular, shared edge endpoints can cause multiple penalties for the same contact.

The offline synthesizer receives relational specifications, cycle or failure witnesses, the typed library, and computation limits. It proposes feature ASTs and a corresponding rule AST. Type checks, expression-size limits, finite-output checks, and a closed execution interface reject invalid proposals. Generated programs cannot invoke the oracle, external services, arbitrary imports, or saved labels at deployment. The LLM proposes distinctions and formulas; certificate checking and program evaluation determine acceptance.

The budgeted loop retains earlier specifications, replays them after each revision, and records unresolved queries as well as successes. Optional acquisition can request comparisons on training instances where retained programs disagree. A disagreement is a query priority, not a certified preference; it enters $\mathcal{D}$ only after the bound checks succeed. Neither feature proposal nor budgeted search is claimed to converge to a consistent or optimal program.

3.5 Joint Selection, Deployment, and Scope

Candidate selection jointly considers specification consistency, complete-schedule quality, and feature cost. Let $s(P)$ be the fraction of certified training requirements satisfied, $q(P)$ the normalized reward on a separate selection set, and $c(P)$ the mean deterministic feature work. The prototype uses the declared selection utility

$$J(P) = q(P) - \lambda(c(P)/c_{\text{ref}} - 1), \quad (10)$$

subject to $s(P) \geq \tau$ and valid interfaces,

where $c_{\text{ref}} > 0$ is the work of the weight-only baseline. Selection reward is normalized by the optimum on the small selection problems where that optimum is computed offline. The targeted variant additionally rejects an exact-alias quotient containing a cycle. Utility ties favour higher schedule quality, lower feature cost, and then earlier candidates. The pilot configuration uses $\tau = 0.75$ and $\lambda = 0.002$; these are declared experimental settings, not theoretical guarantees. All remaining violations are reported, so passing the gate is

not described as full specification satisfaction. No selection score establishes unseen-instance consistency or schedule optimality.

The deterministic work counter includes primitive dispatches, set and membership scans, weight reads, and arithmetic or aggregation work. It is a declared cost proxy, not an exact CPU-instruction count. Actual feature and scoring time and total deployment latency are also recorded. Feature caching is valid only for the graph and residual state on which the feature depends. Offline costs separately record LLM calls, candidate proposals, oracle work, and selection evaluations; zero online calls do not imply zero discovery cost.

After selection, both ϕ and h are frozen. Held-out evaluation separates contact instances and constraint configurations from synthesis and selection, including tests of their joint shift. The same program responds to each current graph within the fixed kernel; no online LLM or optimization-oracle query is allowed. Adaptation therefore means changes in the frozen program’s ranking as its deployable structural inputs change.

Prototype status and limitations. The initial executable prototype uses assistant-proposed candidates and replayable offline artifacts. This is assisted synthesis, not evidence of an autonomous API evolution run. Large-graph certification can be expensive or inconclusive; pairwise graph modelling has a stated scope; the typed library may omit the required distinction; finite evidence can overfit; and feature costs can grow with graph density. A publishable evaluation must retain candidate provenance and measure these limitations rather than equating successful small examples with effective general adaptation.

4 Experimental Protocol

4.1 Data and Split Discipline

The pilot contains 36 pairs in each of training, validation, and test: 12 designed diagnostic pairs, 16 random temporal pairs, and 8 C3-derived local subproblems. Each pair preserves contacts and changes one resource parameter. Generation does not query an oracle or filter by heuristic quality. Contact instances and intervention targets are disjoint across splits; all seeds and graph fingerprints are audited. C3 has nonoverlapping original contact identifiers across three days and disjoint intervention targets, with a shared declared reference configuration. Its source contains 69,923 opportunities; the pilot uses local subproblems capped at 28 contacts and does not measure full-source schedule quality.

Designed probes have 9 or 11 contacts and deliberately produce exact aliases under all nine base features, including both resource-gap inputs. They test the diagnostic and are reported separately. Random temporal contacts use a single-capacity overlap model. C3 retains the original frozen conflict predicates, including legacy satellite overlap semantics, rather than substituting the synthetic model. Graph feasibility is only claimed under the respective model.

Training yields 34 certified pairs: 8 diagnostic reversals, 2 diagnostic preservations, 16 random preservations, and 8 C3 preservations. Two diagnostic ties remain unknown. Comparisons use exact rational arithmetic for saved binary weights and outward-rounded interval endpoints. A 100,000-node branch-and-bound reference finishes exactly for all 72 test contexts. Small pilot certificates use the complete residual region. Progressive local bounds are

implemented, but this pilot does not establish their large-instance certification efficiency.

4.2 Candidates and Controls

The bank contains 48 validated programs: 34 deterministic grid candidates and 14 current-assistant proposals based on saved training evidence. All programs share a kernel and nine base features: weight, duration, degree, conflict-weight sum, maximum conflict weight, compatible-weight sum, two resource gaps, and residual count. Expanded candidates compose the same typed library. Requests, candidates, bounds, selection evaluations, frozen programs, source snapshots, and byte hashes are archived.

We retain weight-only, weight/degree, weighted-conflict normalization, fixed-representation rule search, and joint feature–rule selection. Free-joint selection uses the identical bank, evidence, validation evaluations, and utility, omitting only the contradiction gate. Ungated quality–cost selection and deterministic enumeration are additional controls. Enumeration uses its 34-candidate subpool; it is not equated to an autonomous LLM search budget.

The specification threshold and cost penalty are 0.75 and 0.002. The base representation contains 20 self-loop requirements among 68 training inequalities, so its strict-consistency fraction is at most $48/68 = 0.7059$. This excludes base-only programs from the joint gate. Fixed-representation controls remain evaluated and their violations reported. Consequently this shared-bank ablation cannot establish an independent benefit of contradiction-guided candidate generation.

4.3 Metrics and Reproducibility

Metrics include reward divided by the exact residual optimum, strict consistency, feasibility, deterministic feature work, and measured feature/scoring/deployment time. Every C3 output is also checked using the original source verifier. One frozen program per method is applied to every test instance and configuration without online oracle or LLM calls. Reference computation is offline evaluation instrumentation.

Descriptive intervals resample problem pairs, keeping their configurations together, using 2,000 bootstrap samples and a saved seed. These small heterogeneous samples limit population inference. Timing is a single local observation rather than a controlled hardware benchmark. The audited run re-executes an initial run after audit fixes using unchanged candidates and settings; it is not another independent test. A fully executed notebook verifies saved hashes and calculates the tables and figures.

5 Pilot Results and Further Experiments

The selected program adds $r(v) = \sum_{\{i,j\} \in E[N_R(v)]} \min(w_i, w_j)$ and ranks by

$$w_v - 0.6 \sum_{u \in N_R(v)} w_u + \frac{r(v)}{\max(1, |N_R(v)|)}. \quad (11)$$

This captures redundancy among excluded alternatives, but is a heuristic feature rather than a certified MWIS bound. Overall test reward ratio is 0.9973, compared with 0.9952 for rule-only and 0.9128 for weight-only. All outputs are feasible. C3 ratios are 0.9902

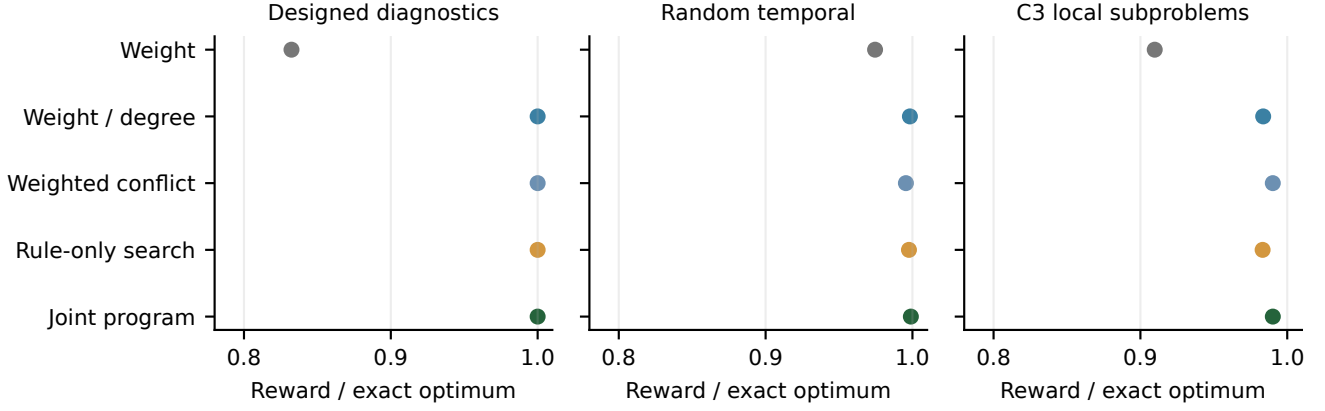


Figure 1: Reward ratios for 12 diagnostic, 16 random, and 8 C3 pairs. The horizontal axis is explicitly restricted. Strong simple rules already perform well; strict relational consistency is a separate metric.

Table 1: Pilot reward divided by exact optimum. Strict counts cover 68 certified inequalities. Designed probes do not estimate workload prevalence.

Method	Diag.	Random	C3	Strict
Weight	0.8324	0.9746	0.9098	44/68
Weight / degree	1.0000	0.9983	0.9836	46/68
Weighted conflict	1.0000	0.9955	0.9902	45/68
Rule-only search	1.0000	0.9976	0.9832	45/68
Joint program	1.0000	0.9989	0.9902	67/68

and 0.9832 for joint and rule-only selection. The paired mean difference over rule-only is 0.00129 on random problems, with descriptive interval $[-0.00037, 0.00384]$, and 0.00695 on C3, with interval $[-0.00158, 0.01784]$. Both include zero; we do not claim an established performance advantage.

Joint selection satisfies 67 of 68 test inequalities, compared with 45 for rule-only, and still violates one certified preference. Some base rules achieve optimal schedules on designed probes despite tied scores for the aliased actions: they can first select peripheral contacts and change the residual state, or use an incidental tie convention. The quotient result proves strict-ranking impossibility at the observed states, not unavoidable complete-schedule loss. Expansion removes observed representation contradictions without guaranteeing a fully consistent rule.

Joint selection uses mean 12,235 feature-work units per test context, versus 8,320 for weight-only and 11,530 for weight/degree. Representation expansion therefore has a measurable cost. Targeted, free-joint, ungated quality-cost, and deterministic enumeration select the same program. The pilot supports an executable representation-repair path, but neither an LLM contribution nor an independent targeted-search advantage.

Frozen-program follow-up. After the pilot, we evaluated the unchanged archived programs on 35 new pairs: 12 diagnostic, 16 random, and 7 C3 pairs; one requested C3 window was empty. These

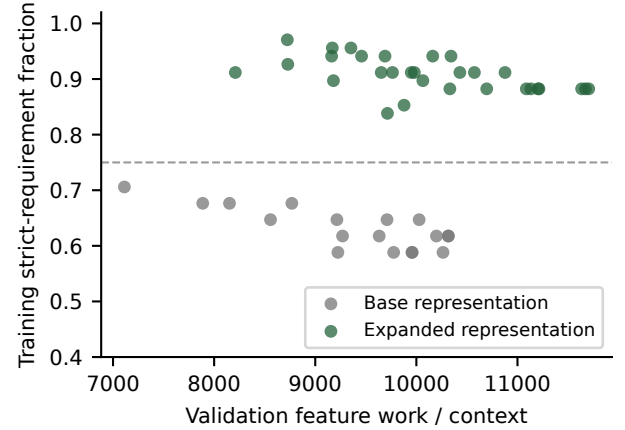


Figure 2: Training consistency versus validation feature work for all 48 candidates. The dashed line is the 0.75 threshold. Work measures complete schedule evaluation and is a declared proxy.

have no graph or original-contact overlap with any pilot split. No program was resynthesized or selected using their outcomes. All 70 references are exact; all 560 method-context schedules are feasible, and all 112 C3 schedules pass the original verifier. The joint program reaches 0.9968 overall, versus 0.9974 for rule-only. Random ratios are 0.9988 versus 0.9973, while C3 ratios are 0.9868 versus 0.9934. Strict consistency remains higher, 61/64 versus 41/64, but three certified inequalities are violated. This reversal of the aggregate reward difference reinforces that representation adequacy and complete-schedule quality must be evaluated separately. The follow-up supports neither uniform reward superiority nor an LLM-specific advantage.

Required next experiments. Independent matched-budget generation runs should compare witness-guided and free feature-rule

search with identical graph information and typed libraries, recording actual model versions, calls, tokens, prompts, failed candidates, and repeated sampling. Further controls should remove preservation, cost selection, or certified labels while retaining comparable acquisition costs. Larger original C3 windows must establish certification success and deployment latency as density grows. These experiments evaluate the central mechanism without replacing it by rule-only aggregate search or treating replay as model evidence.

6 Conclusion

We presented an offline synthesis framework that uses exact contradictions in certified ranking evidence to refine graph representations. Sound conditional bounds, typed composition, cost-aware selection, and a feasible kernel make the mechanism executable and auditable. The pilot exposes input limitations that rule-only repair cannot resolve, but shared-bank controls select the same program and small reward differences do not establish an LLM or targeted-search advantage. Independent synthesis trials and larger held-out workloads are required to test those advantages.

References

- [1] Shengkai Chen, Zhiguang Cao, Jianan Zhou, Yaoxin Wu, Senthilnath Jayavelu, Zhuoyi Lin, Xiaoli Li, and Shili Xiang. 2026. DRAGON: LLM-Driven Decomposition and Reconstruction Agents for Large-Scale Combinatorial Optimization. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 2809–2817. doi:10.65109/SBAY6258
- [2] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *Computer Aided Verification (Lecture Notes in Computer Science, Vol. 1855)*, E. Allen Emerson and A. Prasad Sistla (Eds.). Springer, 154–169. doi:10.1007/10722167_15
- [3] Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. 2024. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 32201–32223. <https://proceedings.mlr.press/v235/liu24bs.html>
- [4] André Grahl Pereira, Augusto B. Corrêa, and Jendrik Seipp. 2026. Property-Guided LLM Program Synthesis for Planning. In *Proceedings of the Fortieth Annual Conference on Neural Information Processing Systems*. arXiv:2605.16142 <https://arxiv.org/abs/2605.16142>
- [5] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical discoveries from program search with large language models. *Nature* 625 (2024), 468–475. doi:10.1038/s41586-023-06924-6
- [6] Xianliang Yang, Ling Zhang, Haolong Qian, Lei Song, and Jiang Bian. 2025. HeurA-genix: Leveraging LLMs for Solving Complex Combinatorial Optimization Challenges. arXiv:2506.15196 [cs.AI] doi:10.48550/arXiv.2506.15196
- [7] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. 2024. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-1381